

Relazione - Versione 8: LLM Parser

Introduzione

La Versione 8 introduce il Parser basato su LLM. Nella Versione 7 il progetto utilizzava già un modello linguistico locale per il routing, mentre l'estrazione degli argomenti era ancora affidata a un Generic Parser rule-based. Nella Versione 8 il Router rimane invariato, ma il Parser utilizza lo stesso modello locale per estrarre argomenti strutturati dalla richiesta dell'utente.

Il modello continua a essere eseguito localmente nel notebook e viene scaricato da Hugging Face. L'obiettivo della versione è sostituire le regole manuali di estrazione con uno step di parsing più flessibile e semantico, mantenendo stabile il resto dell'architettura.

Obiettivo della Versione 8

L'obiettivo principale della Versione 8 è introdurre il parsing degli argomenti basato su modello linguistico.

- LLM Router continua a selezionare il tool più appropriato.
- LLM Parser riceve il tool selezionato, la sua descrizione, i parametri richiesti e la richiesta dell'utente.
- Il parser restituisce argomenti strutturati in formato JSON.
- Planner, Executor, Memory Manager e Agent rimangono invariati.

Cosa cambia rispetto alla Versione 7

La Versione 7 introduceva l'LLM Router, ma continuava a dipendere da un Generic Parser rule-based con funzioni di supporto come l'estrazione dei numeri e dei nomi. La Versione 8 elimina queste funzioni manuali e le sostituisce con un LLM Parser.

Il Tool Registry diventa ancora più importante. Ora supporta entrambi i componenti LLM: il Router usa nomi e descrizioni dei tool, mentre il Parser usa la descrizione del tool selezionato e i parametri richiesti.

Nuovo concetto: LLM Parser

LLM Parser trasforma l'estrazione degli argomenti in un piccolo compito di comprensione del linguaggio. Invece di applicare pattern espliciti, il parser costruisce un prompt che contiene il tool selezionato, la descrizione del tool, i parametri richiesti e la richiesta originale.

Selected tool:
multiplication

Tool description:
Multiply two numbers.

Required parameters:
a, b

User request:
Multiply three by four.

Return only a valid JSON object.

Al modello locale viene chiesto di restituire solo JSON. Il codice Python legge poi l'output del modello, estrae il frammento JSON e lo converte in un dizionario Python.

Uso del Tool Registry nel parsing

Il Tool Registry rimane la fonte condivisa e unica di verità per tutti i tool e i metadati. Nella Versione 8 il Parser legge la descrizione del tool selezionato e l'elenco dei parametri richiesti prima di costruire il prompt.

```
tool_info = tool_registry.get(tool_name)
parameters = ", ".join(tool_info["parameters"])
```

Questo significa che la logica di parsing non è più codificata manualmente per ogni tool. Il parser adatta il prompt in base ai metadati registrati nel Tool Registry.

Normalizzazione degli argomenti

Dopo il parsing rimane un piccolo passaggio di normalizzazione. Il suo scopo non è decidere il significato della richiesta, ma semplicemente convertire valori come "2" in 2 e "2.5" in 2.5 quando possibile.

```
def normalize_argument(value):
    if isinstance(value, str):
        if value.isdigit():
            return int(value)
        try:
            return float(value)
        except ValueError:
            return value
    return value
```

L'estrazione semantica continua comunque a essere svolta dal modello linguistico. La normalizzazione garantisce solo che l'Executor riceva un formato dati pulito.

Architettura completa nella Versione 8

Il flusso di esecuzione diventa:

```
Request -> LLM Router -> Planner -> LLM Parser -> Executor -> Memory Manager ->
Memory
                                     |
                                     v
                                   Output
```

Stato dell'agente nella Versione 8

La struttura dello stato resta la stessa della Versione 7:

```
{
  "request": "Multiply three by four.",
  "plan": {
    "tool": "multiplication",
    "reason": "Multiply two numbers.",
    "needs_arguments": ["a", "b"]
  },
  "action": "multiplication",
  "arguments": {"a": 3, "b": 4},
  "result": 12
}
```

Test

I test confermano sia la compatibilità con la Versione 7 sia il nuovo comportamento di parsing:

```
clear_memory()
tool_registry
agent("What is 2 + 3?")
agent("Hello, my name is luca.")
agent("What is the weather in Rome?")
agent("Could you calculate the product of 2 and 3?")
agent("Multiply three by four.")
get_memory()
get_last_interaction()
```

- "Could you calculate the product of 2 and 3?" verifica che l'LLM Router continui a comprendere l'intento in modo semantico.
- "Multiply three by four." verifica il miglioramento chiave della Versione 8: l'LLM Parser sa estrarre argomenti numerici espressi in parole e non solo in cifre.
- "Hello, my name is luca." verifica che anche l'estrazione del nome venga gestita dall'LLM Parser.
- La richiesta sul meteo verifica che le richieste non supportate continuino a restituire nessun tool.

Cosa insegna la Versione 8

La Versione 8 insegna il secondo uso pratico di un modello linguistico all'interno dell'architettura dell'agente: l'estrazione semantica degli argomenti. Il sistema ora usa un LLM sia per decidere quale tool usare sia per capire quali valori devono essere passati a quel tool.

Limiti

- il piccolo modello locale può ancora commettere errori di parsing;
- il Planner rimane rule-based e continua a creare un piano a singolo step;
- il Tool Registry rimane in memoria e la registrazione è manuale;
- la memoria rimane una semplice lista;
- l'agente continua a eseguire una sola azione alla volta.

Conclusione

La Versione 8 è un'evoluzione naturale della Versione 7. Il Router continua a selezionare i tool in modo semantico e il nuovo LLM Parser ora estrae in modo semantico anche gli argomenti. Il resto dell'architettura rimane volutamente stabile per rendere il nuovo concetto facile da isolare e comprendere.

Il test riuscito con "Multiply three by four." dimostra il beneficio principale della versione: non sono più necessarie regole manuali per interpretare argomenti espressi in linguaggio più naturale. Il passo successivo sarà la Versione 9, che introdurrà l'esecuzione multi-step.